Finally, we bundle install (line 9) to install the gems we need for our Rails project.

Now that it makes more sense, let's go ahead and actually create this Dockerfile. First let's make sure we're in the top-level (root) directory of our Rails app:

```
$ ls
Gemfile     Rakefile  bin     config.ru  lib  package.json  storage  vendor
README.md   app       config  db         log  public        tmp
```

Then crack open your editor of choice and create a file called Dockerfile with the contents as shown. I'd highly recommend typing it in by hand rather than copying and pasting—when learning a new skill, physically typing things out helps to cement it in your mind and build up your muscle memory.

With our swanky Dockerfile in hand, let's turn our attention to how we use it to create an actual image.

Building Our Image

The process of generating an image from a Dockerfile is called *building an image*. We do that with the docker build command, which has the following format:

```
$ docker build [options] path/to/build/directory
```

In our case, you should already still be in the directory containing our Dockerfile and project files so we can use a single dot to indicate the current directory. Let's give it a go:

```
$ docker build .
Sending build context to Docker daemon  138.8kB
Step 1/6 : FROM ruby:2.6
 ---> f28a9e1d0449
Step 2/6 : RUN apt-get update -yqq
 ---> Running in 29677ed71d2b
Removing intermediate container 29677ed71d2b
 ---> 761da319d69a
«...»
Step 6/6 : RUN bundle install
 ---> Running in 4550030ac412
«...»
Bundle complete! 15 Gemfile dependencies, 68 gems now installed.
Bundled gems are installed into `/usr/local/bundle`
«...»
Removing intermediate container 4550030ac412
 ---> a1df0eddba18
Successfully built a1df0eddba18
```

Wow, that's quite a lot of output. What is actually happening?